

```
import pandas as pd
df = pd.read_csv("/content/sample_data/Cars.csv")

print(df.isnull().sum())
```

Make	0
Model	0
Year	0
Engine Fuel Type	3
Engine HP	69
Engine Cylinders	30
Transmission Type	0
Driven_Wheels	0
Number of Doors	6
Market Category	3742
Vehicle Size	0
Vehicle Style	0
highway MPG	0
city mpg	0
Popularity	0
MSRP	0

dtype: int64

```
missing_percentage = (df.isnull().sum()/len(df))*100
print(missing_percentage)
```

Make	0.000000
Model	0.000000
Year	0.000000
Engine Fuel Type	0.025180
Engine HP	0.579151
Engine Cylinders	0.251805
Transmission Type	0.000000
Driven_Wheels	0.000000
Number of Doors	0.050361
Market Category	31.408427
Vehicle Size	0.000000
Vehicle Style	0.000000
highway MPG	0.000000
city mpg	0.000000
Popularity	0.000000
MSRP	0.000000

dtype: float64

```
df['Engine HP'].fillna(df['Engine HP'].mean(),inplace = True)
```

/tmp/ipython-input-379821912.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values

always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Engine HP'].fillna(df['Engine HP'].mean(),inplace = True)
```

```
print(df.isnull().sum())
```

```
Make          0
Model         0
Year          0
Engine Fuel Type    3
Engine HP        0
Engine Cylinders   30
Transmission Type  0
Driven_Wheels     0
Number of Doors    6
Market Category  3742
Vehicle Size      0
Vehicle Style     0
highway MPG       0
city mpg          0
Popularity        0
MSRP             0
dtype: int64
```

```
df['Engine Cylinders'].fillna(df['Engine Cylinders'].median(), inplace = True)
```

/tmp/ipython-input-3154144365.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Engine Cylinders'].fillna(df['Engine Cylinders'].median(), inplace = True)
```

```
print(df.isnull().sum())
```

Make	0
Model	0
Year	0
Engine Fuel Type	3
Engine HP	0
Engine Cylinders	0
Transmission Type	0
Driven_Wheels	0
Number of Doors	6
Market Category	3742
Vehicle Size	0
Vehicle Style	0
highway MPG	0
city mpg	0
Popularity	0
MSRP	0

dtype: int64

```
df['Engine Fuel Type'].fillna(df['Engine Fuel Type'].mode()
[0],inplace=True)
```

/tmp/ipython-input-3709055531.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Engine Fuel Type'].fillna(df['Engine Fuel Type'].mode()
[0],inplace=True)
```

```
print(df.isnull().sum())
```

Make	0
Model	0
Year	0
Engine Fuel Type	0
Engine HP	0
Engine Cylinders	0
Transmission Type	0
Driven_Wheels	0
Number of Doors	6
Market Category	3742
Vehicle Size	0

```
Vehicle Style      0
highway MPG        0
city mpg           0
Popularity         0
MSRP               0
dtype: int64
```

```
df['Number of Doors'].fillna(method = 'ffill',inplace = True)
```

/tmp/ipython-input-524543580.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Number of Doors'].fillna(method = 'ffill',inplace = True)
/tmp/ipython-input-524543580.py:1: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
```

```
df['Number of Doors'].fillna(method = 'ffill',inplace = True)
```

```
df['Market Category'].fillna(method = 'bfill',inplace = True)
```

/tmp/ipython-input-1453890980.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Market Category'].fillna(method = 'bfill',inplace = True)
/tmp/ipython-input-1453890980.py:1: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
```

```
df['Market Category'].fillna(method = 'bfill',inplace = True)
```

```
print(df.isnull().sum())
```

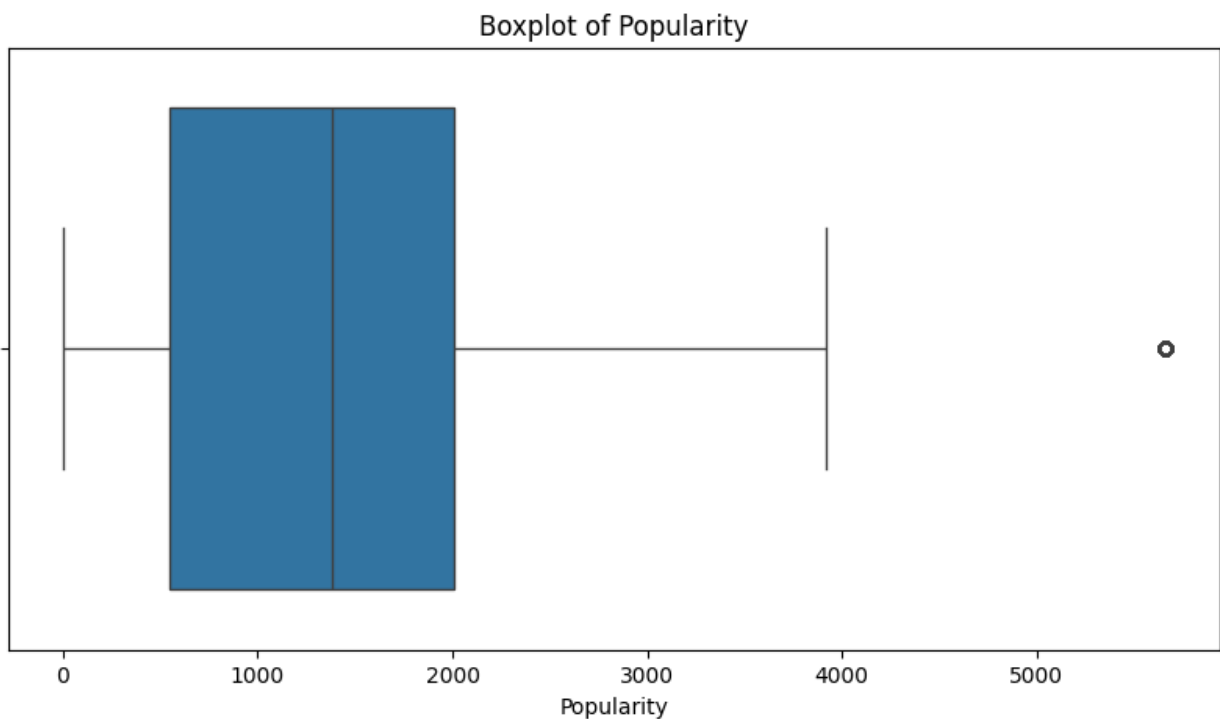
```

Make          0
Model         0
Year          0
Engine Fuel Type  0
Engine HP     0
Engine Cylinders 0
Transmission Type 0
Driven_Wheels  0
Number of Doors 0
Market Category 0
Vehicle Size   0
Vehicle Style  0
highway MPG    0
city mpg       0
Popularity     0
MSRP          0
dtype: int64

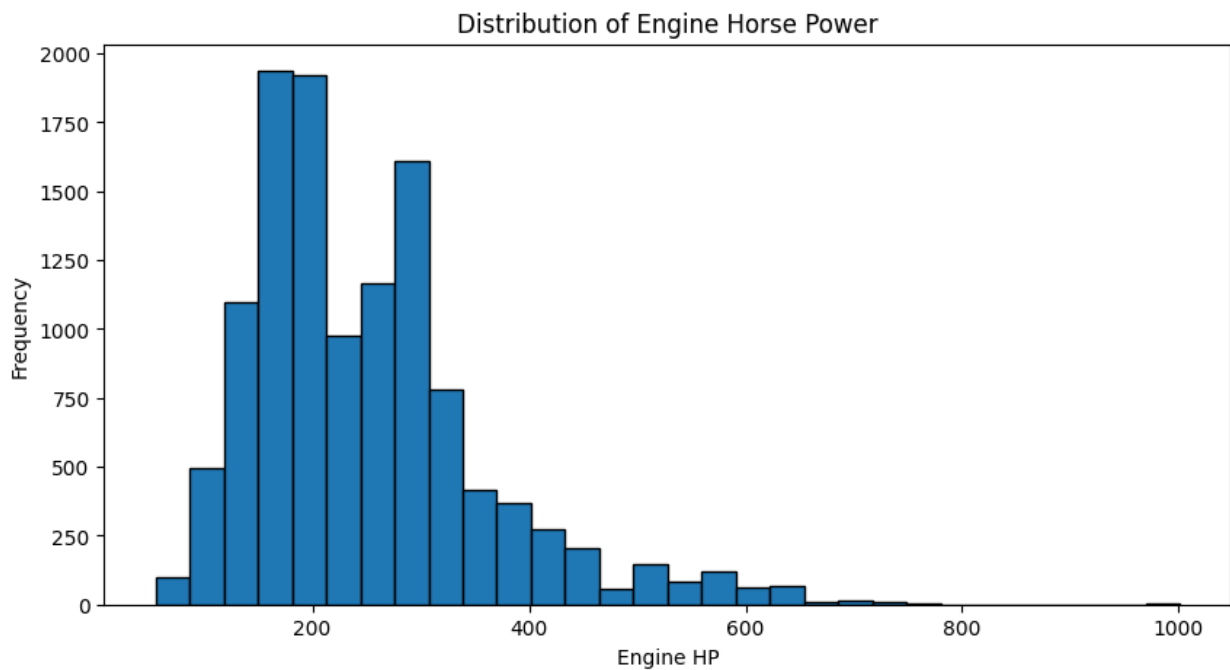
import matplotlib.pyplot as plt
import seaborn as sns

# Box Plot
plt.figure(figsize = (10,5))
sns.boxplot( x = df['Popularity'])
plt.title("Boxplot of Popularity")
plt.show()

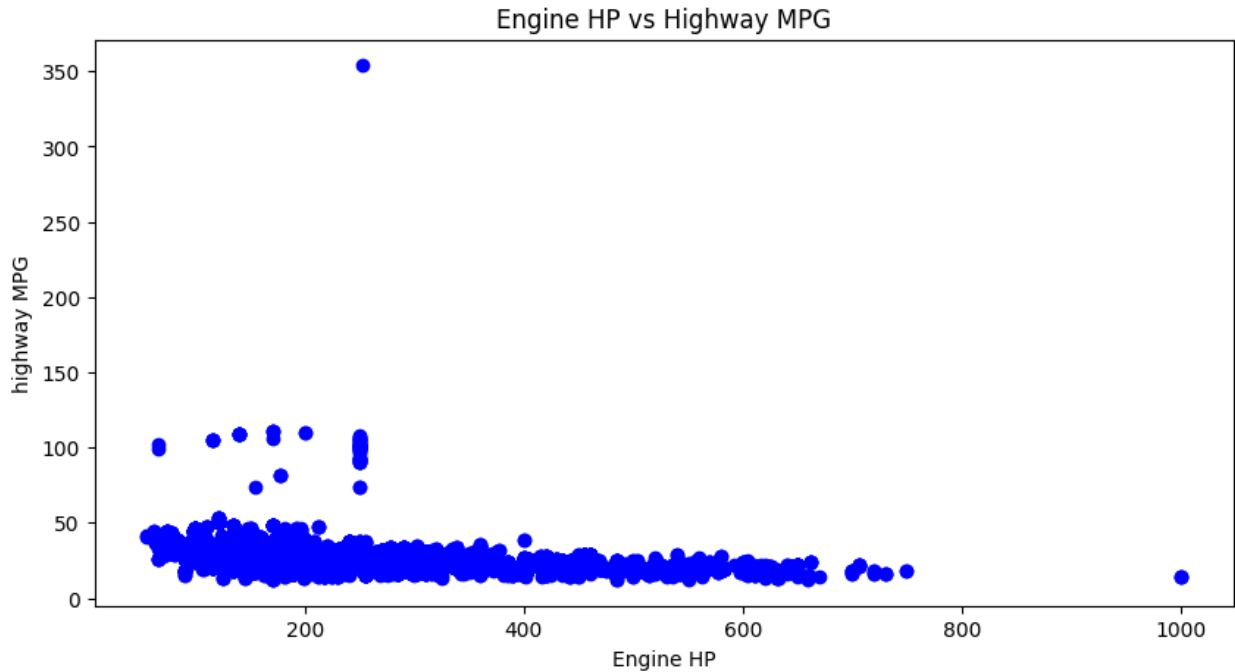
```



```
# Histogram
plt.figure(figsize=(10,5))
plt.hist(df['Engine HP'], bins = 30 , edgecolor = 'black')
plt.title("Distribution of Engine Horse Power")
plt.xlabel("Engine HP")
plt.ylabel("Frequency")
plt.show()
```

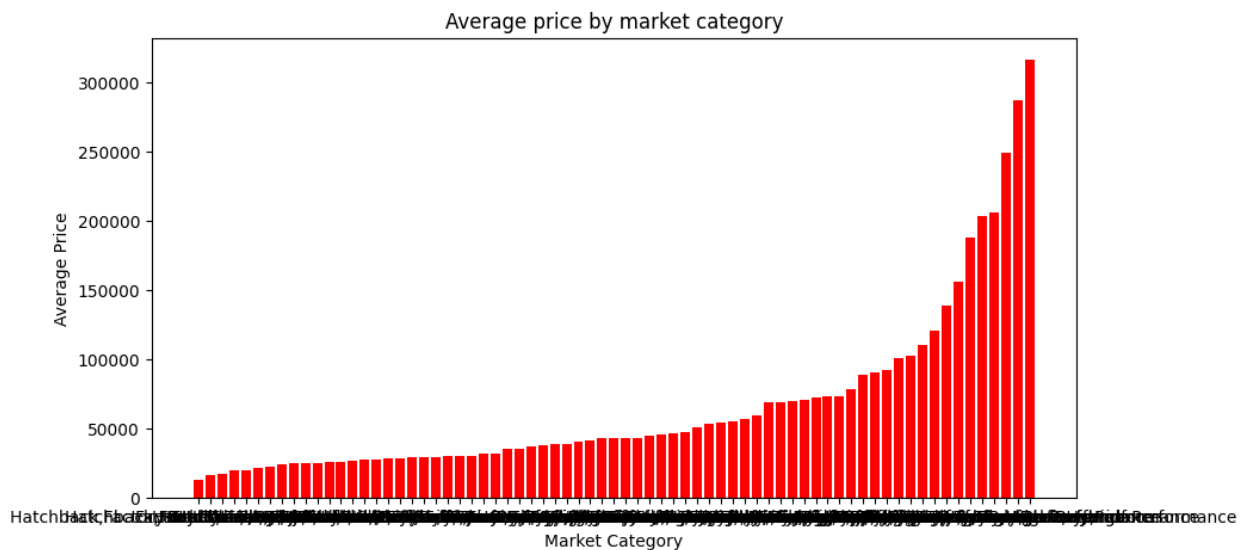


```
# Scatter Plot
plt.figure(figsize = (10,5))
plt.scatter(df['Engine HP'], df['highway MPG'], color = 'blue')
plt.title("Engine HP vs Highway MPG")
plt.xlabel("Engine HP")
plt.ylabel("highway MPG")
plt.show()
```

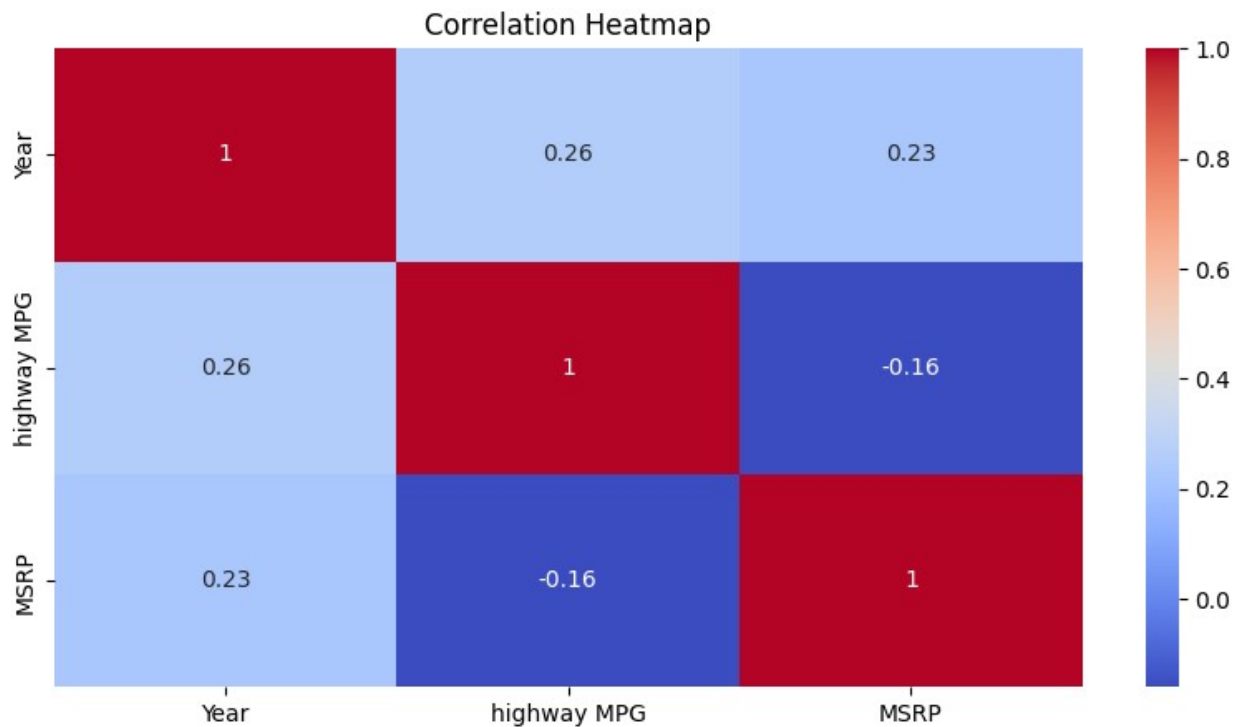


```
# Bar Plot
avg_price = df.groupby("Market Category")["MSRP"].mean().sort_values()

plt.figure(figsize=(10,5))
plt.bar(avg_price.index, avg_price.values, color = 'red')
plt.title("Average price by market category")
plt.xlabel("Market Category")
plt.ylabel("Average Price")
plt.show()
```



```
# Heat Map
plt.figure(figsize = (10,5))
sns.heatmap(df[["Year","highway MPG","MSRP"]].corr(), annot =True,
cmap = "coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



```
# Line Plot
yearly_price = df.groupby("Year")["MSRP"].mean()

plt.figure(figsize=(10,5))
plt.plot(yearly_price.index, yearly_price.values, marker = 'o',color =
'green')
plt.title("Average Price over a year ")
plt.xlabel("Year")
plt.ylabel("Average Price")
plt.grid(True)
plt.show()
```